

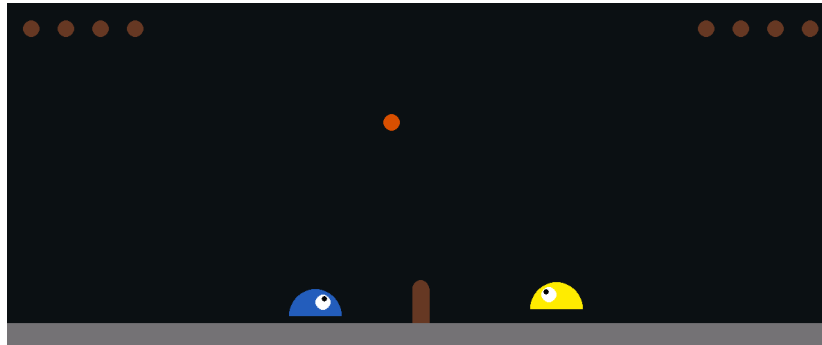
Uczenie ze wzmocnieniem w środowisku Slime Volleyball - Sprawozdanie

Robert Mesek

7 czerwca 2026

1 Wprowadzenie

Przedmiotem projektu jest implementacja i analiza systemu uczącego się w środowisku *SlimeVolley-v0* [1], będącym portem klasycznej, dwuwymiarowej gry w siatkówkę. Aby umożliwić integrację z nowoczesnymi bibliotekami, środowisko zostało zmodernizowane przy pomocy narzędzi AI, co zapewniło kompatybilność z aktualnym standardem API Gymnasium [3].



Rysunek 1: Wizualizacja środowiska *SlimeVolley-v0*.

Celem agenta jest przebicie piłki przez siatkę w taki sposób, aby wylądowała ona na polu przeciwnika, co skutkuje utratą przez niego jednego z pięciu początkowych żyć. Epizod kończy się w momencie, gdy którykolwiek z graczy straci wszystkie życia, lub po upływie limitu 3000 kroków czasowych. Za każde życie odebrane przeciwnikowi agent otrzymuje nagrodę dodatnią (+1), natomiast utrata własnego punktu wiąże się z analogiczną karą (-1).

Agent podejmuje decyzje na podstawie obserwacji wektorów pozycji i prędkości obiektów w środowisku, a jego zadaniem jest opanowanie optymalnej strategii poruszania się. W ramach zaproponowanego podejścia proces uczenia realizowany jest poprzez bezpośrednią rywalizację z wcześniej wytrenowaną, ekspercką polityką bazową.

2 Koncepcja rozwiązania

Do rozwiązania problemu zastosowano podejście oparte na uczeniu ze wzmocnieniem (ang. *Reinforcement Learning*), z wykorzystaniem algorytmu PPO (Proximal Policy Optimization) dostarczanego przez bibliotekę AgileRL [4]. Uczenie agenta jest dodatkowo wspomagane ewolucyjną optymalizacją hiperparametrów (mutacje architektury i parametrów RL).

2.1 Przestrzeń stanu i akcji

Stan środowiska (obserwacja) jest reprezentowany przez 12-wymiarowy wektor zawierający współrzędne położenia oraz wektory prędkości obu graczy i piłki z

perspektywy agenta: x_{ag} , y_{ag} , vx_{ag} , vy_{ag} , x_{ball} , y_{ball} , vx_{ball} , vy_{ball} , x_{opp} , y_{opp} , vx_{opp} , vy_{opp} . Przestrzeń akcji to natomiast wektor `MultiBinary(3)`, co odpowiada jednoczesnemu sterowaniu ruchem w lewo, w prawo oraz skokiem.

2.2 Składowe nagród

Testy wykazały, że przyznawanie agentowi ciągłych nagród za samo odbijanie piłki prowadzi do wpadania w minima lokalne. Agent uczył się minimalizować kary, zamiast dążyć do zdobycia punktu. Zmodyfikowano więc środowisko, tworząc wariant `SlimeVolleyShaped-v0`. Wprowadzono w nim bardzo małą nagrodę o wartości 0.01 za przebicie piłki na połowę oponenta, co promuje aktywną grę. Ponieważ agenci potrafili zbyt długo podbijać piłkę po swojej stronie dodano zasadę, w której jeśli agent dotknie piłki więcej niż trzy razy z rzędu jest to traktowane jak strata punktu. Co ciekawe, wbudowana polityka bazowa nie знаła tej zasady, ale jej styl gry i tak opierał się na wczesnych przebiciach nad siatką.

2.3 Ekspercka polityka bazowa

Przeciwnikiem w trakcie treningu była wbudowana polityka ekspercka [2]. Jest to mała rekurencyjna sieć neuronowa składająca się ze 120 parametrów. Została ona wcześniej wytrenowana od podstaw metodą uczenia przez grę przeciwko samemu sobie przy użyciu algorytmów genetycznych. Agenci rozgrywali ze sobą krótkie symulowane mecze bez żadnych zaprogramowanych heurystyk, optymalizując strategię wyłącznie na podstawie wygranych i przegranych.

3 Architektura systemu

System opiera się na algorytmie PPO, który jest stabilną metodą gradientową optymalizującą politykę aktora w trakcie gry. Działa on dobrze w tego typu problemach dzięki mechanizmowi przycinania funkcji celu, co zapobiega zbyt agresywnym aktualizacjom wag sieci podczas pojedynczego kroku uczenia. Parametry startowe, w tym współczynnik uczenia równy $3 \cdot 10^{-4}$ oraz rozmiar paczki danych wielkości 512, zostały dobrane na podstawie dokumentacji środowiska i własnych testów.

Ważnym elementem systemu jest wykorzystanie algorytmów ewolucyjnych. Podczas treningu na agentów w populacji nakładane są mutacje. Parametry uczenia mogą ulec zmianie z prawdopodobieństwem 30 procent, natomiast architektura samej sieci neuronowej ma szansę na zmianę struktury lub dodanie nowej warstwy. Pozwala to na automatyczne dostosowywanie modelu.

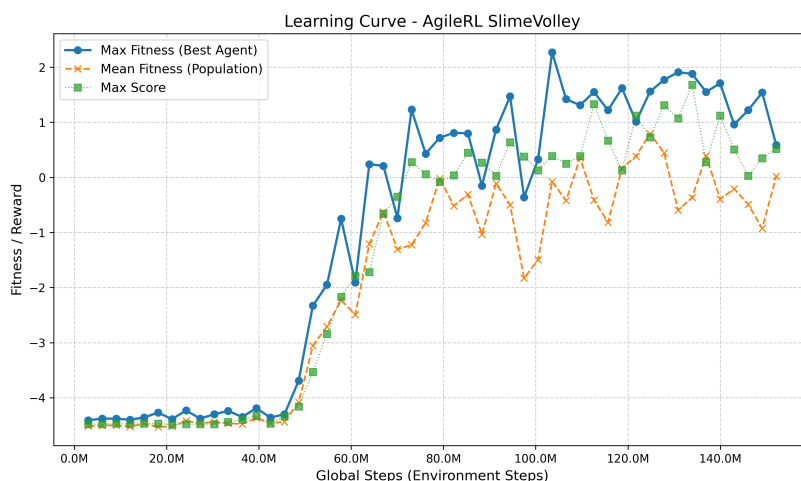
4 Wyniki eksperymentów

Trening wymagał przeprowadzenia dużej liczby kroków symulacji, przekraczającej 23 miliony iteracji dla całej populacji modeli. Wyniki i wykresy pokazują, że

początkowa faza uczenia jest bardzo trudna, jednak model ostatecznie opanował grę w siatkówkę i zaczął regularnie pokonywać bazowego eksperta.

Najlepszy z wytrenowanych modeli zmodyfikował i powiększył swoją początkową architekturę sieci. Powstał w niej wspólny moduł przetwarzający dane wejściowe, składający się z trzech warstw liniowych o rozmiarach 64, 80 i 32 neuronów z zastosowaniem aktywacji ReLU. Optymalizacja ewolucyjna dostosowała też hiperparametry algorytmu: współczynnik uczenia spadł do około $9.8 \cdot 10^{-5}$, przetwarzana paczka danych wzrosła do 564, a rozmiar kroku środowiska ustalił się na poziomie 648.

Początkowe próby zrealizowania nauki zupełnie od zera przez grę własnych agentów przeciwko sobie okazały się na tym etapie zbyt czasochłonne obliczeniowo z wykorzystaniem metod jak IPPO (Independent Proximal Policy Optimization) dostępnych w bibliotece AgileRL. Możliwym kierunkiem na dalszy rozwój projektu byłoby zastąpienie oryginalnej polityki bazowej wytrenowanym agentem i ponowne uruchomienie procesu uczenia.



Rysunek 2: Przebieg procesu uczenia algorytmu PPO z biblioteki AgileRL w środowisku `SlimeVolleyShaped-v0`.

Literatura

- [1] David Ha. Slime Volleyball Gym Environment. URL: <https://github.com/hardmaru/slimevolleygym>.
- [2] David Ha. Neural slime volleyball. *blog.otoro.net*, 2015. URL: <https://blog.otoro.net/2015/03/28/neural-slime-volleyball/>.
- [3] Mark Towers, Ariel Kwiatkowski, Jordan K Terry, John U. Balis, Gianluca de Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Jin Shen Tan, and Omar G. Younis. Gymnasium: A Standard Interface for Reinforcement Learning Environments. URL: <https://github.com/Farama-Foundation/Gymnasium>.
- [4] Nicholas Ustaran-Anderegg, Michael Pratt, and Jaime Sabal-Bermudez. AgileRL. URL: <https://github.com/AgileRL/AgileRL>.